MERN STACK

E-Commerce Web App

Roman Urdu Mein — Zero Se Hero Tak

CHAPTER 9 | FRONTEND PAGES | GAP FIX INCLUDED | ROMAN URDU

CHAPTER 9

Frontend Pages — Complete ShopEase UI

Chapter 8 ke gaps fix + saare pages complete + working app!

Pehle Chapter 8 Ke Gaps Fix Honge

Chapter 8 mein 2 cheezein adhoori reh gayi thi: (1) ProductScreen ka Add to Cart button Redux se connect nahi tha. (2) CartScreen.jsx file likhi hi nahi gayi thi. Yeh dono Section 1 aur 2 mein fix honge — phir naye pages aayenge.

Is Chapter Mein Aap Seekhenge:

- Chapter 8 Gap Fix — ProductScreen ka Add to Cart Redux se connect karna
- Chapter 8 Gap Fix — CartScreen.jsx poori file likhna
- HomeScreen — search + filter + pagination ke saath complete
- SearchBox Component — keyword search karna
- ProductCard — updated, refined, fully working
- Rating Component — star display banana

- ProductScreen — detail page with reviews
- Paginate Component — page navigation
- Filter Component — category aur price filter
- Final File List — har file ka poora reference

TABLE OF CONTENTS

- ■ 7.2 Paginate.jsx complete
- ■ 7.3 HomeScreen mein use karna

8.0 ProductScreen — Complete

- ■ 8.1 Product fetch (Redux thunk)
- ■ 8.2 Image, naam, price, stock
- ■ 8.3 Quantity selector
- ■ 8.4 Add to Cart (Redux fixed!)
- ■ 8.5 Reviews section display

9.0 Final File Structure

- ■ 9.1 Saari files ka complete list
- ■ 9.2 Kaunsi file kahan rakhen
- ■ 9.3 Import chain — sab connected

10.0 Summary & Practice

- ■ 10.1 Chapter recap
- ■ 10.2 Practice exercises
- ■ 10.3 Chapter 10 preview

SECTION 1 — Gap Fix: ProductScreen

■ CHAPTER 8 KA GAP FIX

Chapter 8 mein ProductScreen ka 'Add to Cart' button sirf URL change karta tha — Redux cartSlice se connected nahi tha. Yeh abhi fix karte hain.

1.0 — Pehle Kya Tha (Problem)

```
// ■ Chapter 8 mein — GALAT — Cart mein kuch add nahi hota tha
const addToCartHandler = () => {
  // Yeh sirf URL change karta tha — Redux store khaali rehta tha
  navigate(`/cart?id=${product._id}&qty=${quantity}`);
};
```

1.1 — Ab Sahi Karte Hain

ProductScreen mein **useDispatch** import karo aur **addItem** action dispatch karo — Redux cartSlice se:

■ src/screens/ProductScreen.jsx — UPDATED

```
import { useState, useEffect }          from 'react';
import { useParams, useNavigate, Link } from 'react-router-dom';
import { useDispatch, useSelector }      from 'react-redux';

// Redux thunk — product fetch karne ke liye
import { fetchProductById,
         selectSelectedProduct,
         selectProductsLoading,
         selectProductsError } from '../store/productSlice';

// Cart action — Redux se
import { addItem } from '../store/cartSlice';

function ProductScreen() {
  const { id }   = useParams();          // URL se product ID lo
  const dispatch = useDispatch();
```

```
  const navigate = useNavigate();

  // Redux store se product data lo
  const product = useSelector(selectSelectedProduct);
  const loading = useSelector(selectProductsLoading);
  const error   = useSelector(selectProductsError);

  // Quantity — sirf local state mein (cart mein jaane se pehle)
  const [quantity, setQuantity] = useState(1);

  // Component mount hone pe product fetch karo
  useEffect(() => {
    dispatch(fetchProductById(id));
  }, [dispatch, id]);
  // id dependency mein hai — URL badlne pe dobara fetch hoga

  // ■ FIXED — Ab Redux cartSlice mein item add hoga
  const addToCartHandler = () => {
    dispatch(addItem({
      _id:      product._id,
      name:     product.name,
      image:    product.images?.[0] || '/images/default.jpg',
      price:    product.price,
      stock:    product.stock,
      quantity: quantity,   // User ne jo quantity choose ki
    }));
    navigate('/cart');  // Cart page pe le jao
  };

  // Loading aur Error states
  if (loading) return (
    <p style={{ color: '#38BDF8', textAlign: 'center', marginTop: '40px'
}}>
      Loading product...
    </p>
  );

  if (error) return (
```

```
      <div style={{ textAlign: 'center', marginTop: '40px' }}>
        <p style={{ color: '#FF6B6B', marginBottom: '16px' }}>{error}</p>
        <Link to='/' style={{ color: '#38BDF8' }}>Home pe wapas jao</Link>
      </div>
    );

    if (!product) return null;  // Data aane tak kuch mat dikhao

    return (
      <div style={{ maxWidth: '1100px', margin: '0 auto', padding: '24px' }
}>

        {/* Back button */}
        <Link to='/' style={{ color: '#8B949E', textDecoration: 'none',
          display: 'inline-block', marginBottom: '20px' }}>
          &larr; Wapas Products Pe
        </Link>

        {/* Main Grid — image left, detail right */}
        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr',
          gap: '40px', alignItems: 'start' }}>

          {/* LEFT — Product Image */}
          <img
            src={product.images?.[0] || '/images/default.jpg'}
            alt={product.name}
            style={{ width: '100%', borderRadius: '12px',
              border: '1px solid #30363D' }}
          />

          {/* RIGHT — Product Details */}
          <div>
            {/* Naam */}
            <h1 style={{ color: '#E6EDF3', fontSize: '24px',
              marginBottom: '12px' }}>
              {product.name}
            </h1>

            {/* Price */}
```

```
<p style={{ color: '#56CF8A', fontSize: '28px',
  fontWeight: 'bold', marginBottom: '12px' }}>
  Rs. {product.price?.toLocaleString()}
</p>

{/* Category */}
<p style={{ color: '#8B949E', fontSize: '13px',
  marginBottom: '16px' }}>
  Category: {product.category}
</p>

{/* Description */}
<p style={{ color: '#C9D1D9', lineHeight: '1.6',
  marginBottom: '20px' }}>
  {product.description}
</p>

{/* Stock Status */}
<p style={{
  color:        product.stock > 0 ? '#56CF8A' : '#FF6B6B',
  fontWeight:   'bold',
  marginBottom: '20px'
}}>
  {product.stock > 0
    ? `In Stock — ${product.stock} bache hain`
    : 'Out of Stock'}
</p>

{/* Quantity Selector — sirf tab dikhao jab in stock ho */}
{product.stock > 0 && (
  <div style={{ display: 'flex', alignItems: 'center',
    gap: '12px', marginBottom: '20px' }}>
    <span style={{ color: '#8B949E' }}>Quantity:</span>

    <button
      onClick={() => setQuantity(q => Math.max(1, q - 1))}
      style={{ background: '#1C2128', color: 'white',
```

```
                  border: '1px solid #30363D', borderRadius: '6px',
                  padding: '6px 14px', cursor: 'pointer', fontSize: '18px
' }}
              > - </button>
              <span style={{ color: 'white', fontSize: '18px',
                minWidth: '30px', textAlign: 'center' }}>
                {quantity}
              </span>
              <button
                onClick={() => setQuantity(q => Math.min(product.stock, q
 + 1))}
                style={{ background: '#1C2128', color: 'white',
                  border: '1px solid #30363D', borderRadius: '6px',
                  padding: '6px 14px', cursor: 'pointer', fontSize: '18px
' }}
              > + </button>
              {/* Math.min(stock, q+1) — stock se zyada nahi ho sakti */}
            </div>
          )}
          {/* Add to Cart Button */}
          <button
            onClick={addToCartHandler}
            disabled={product.stock === 0}
            style={{
              background:   product.stock > 0 ? '#38BDF8' : '#30363D',
              color:        product.stock > 0 ? '#0D1117' : '#8B949E',
              border:       'none',
              borderRadius: '8px',
              padding:      '14px 32px',
              fontSize:     '16px',
              fontWeight:   'bold',
              cursor:       product.stock > 0 ? 'pointer' : 'not-allowed
',
              width:        '100%',
```

```
              }}
            >
              {product.stock > 0 ? 'Add to Cart' : 'Out of Stock'}
            </button>
          </div>
        </div>
      </div>
    );
  }

  export default ProductScreen;
```

■ TIP

Math.min(product.stock, q + 1) — yeh isliye hai ke user stock se zyada quantity select na kar sake. Agar stock 3 hai toh +4 ya +5 nahi hoga. Math.max(1, q - 1) — quantity 0 ya negative nahi hogi!

SECTION 2 — Gap Fix: CartScreen

■ CHAPTER 8 KA GAP FIX

CartScreen.jsx Chapter 8 mein kabhi likhi hi nahi gayi thi — sirf imports aur routes mein naam tha. Yeh poori file ab yahan milegi.

2.0 — CartScreen Ka Plan

CartScreen mein yeh sab hona chahiye:

Feature	Kya Karega?
Items list	Har cart item — image, naam, price, quantity dikhana
Quantity update	+ ya - se quantity badalna — Redux dispatch
Remove button	X se item hatana — Redux dispatch
Empty cart message	Agar cart khaali — message aur shop link
Order Summary	Items total, tax, shipping, grand total
Checkout button	Logged in ho toh checkout, nahi toh login

■ src/screens/CartScreen.jsx [NEW FILE]

```
import { useDispatch, useSelector } from 'react-redux';
import { Link, useNavigate }        from 'react-router-dom';

// Cart slice se actions aur selectors
import { removeItem, updateQuantity,
         selectCartItems, selectTotalItems,
         selectTotalPrice } from '../store/cartSlice';

// Auth slice se login check
import { selectIsLoggedIn } from '../store/authSlice';

function CartScreen() {
  const dispatch    = useDispatch();
  const navigate    = useNavigate();
```

```
// Redux store se cart data lo
const cartItems   = useSelector(selectCartItems);
const totalItems  = useSelector(selectTotalItems);
const totalPrice  = useSelector(selectTotalPrice);
const isLoggedIn  = useSelector(selectIsLoggedIn);

// Checkout handler
const checkoutHandler = () => {
  if (isLoggedIn) {
    navigate('/checkout');   // Chapter 10 mein banayenge
  } else {
    navigate('/login');      // Login karo pehle
  }
};

// ■■ CART KHAALI HAI ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
if (cartItems.length === 0) {
  return (
    <div style={{ textAlign: 'center', padding: '60px 20px' }}>
      <p style={{ fontSize: '60px', marginBottom: '16px' }}>■</p>
      <h2 style={{ color: '#E6EDF3', marginBottom: '8px' }}>
        Aapka Cart Khaali Hai
      </h2>
      <p style={{ color: '#8B949E', marginBottom: '24px' }}>
        Koi product abhi cart mein nahi hai
      </p>
      <Link to='/' style={{
        background: '#38BDF8', color: '#0D1117',
        padding: '12px 28px', borderRadius: '8px',
        textDecoration: 'none', fontWeight: 'bold'
      }}>
        Shopping Karo
      </Link>
    </div>
  );
```

```
  }

  // ■■ CART MEIN ITEMS HAIN ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  // Tax aur shipping calculate karo
  const taxPrice      = totalPrice * 0.05;          // 5% tax
  const shippingPrice = totalPrice > 2000 ? 0 : 150; // 2000+ free shippi
ng
  const grandTotal    = totalPrice + taxPrice + shippingPrice;

  return (
    <div style={{ maxWidth: '1100px', margin: '0 auto', padding: '24px' }
}>
      <h1 style={{ color: '#E6EDF3', marginBottom: '24px' }}>
        Shopping Cart ({totalItems} items)
      </h1>

      {/* Two column layout */}
      <div style={{ display: 'grid',
        gridTemplateColumns: '2fr 1fr', gap: '24px' }}>

        {/* ■■ LEFT: Cart Items ■■■■■■■■■■■■■■■■■■■■■■■■ */}
        <div>
          {cartItems.map(item => (
            <div key={item._id} style={{
              display:       'flex',
              gap:           '16px',
              alignItems:    'center',
              background:    '#161B22',
              border:        '1px solid #30363D',
              borderRadius:  '10px',
              padding:       '16px',
              marginBottom:  '12px'
            }}>

              {/* Product Image */}
              <img src={item.image} alt={item.name}
                style={{ width: '80px', height: '80px',
                  objectFit: 'cover', borderRadius: '6px' }} />
```

```
                {/* Product Name */}
                <div style={{ flex: 1 }}>
                  <Link to={`/product/${item._id}`}
                    style={{ color: '#E6EDF3', textDecoration: 'none',
                      fontWeight: 'bold' }}>
                    {item.name}
                  </Link>
                  <p style={{ color: '#56CF8A', marginTop: '4px' }}>
                    Rs. {item.price?.toLocaleString()}
                  </p>
                </div>

                {/* Quantity Controls */}
                <div style={{ display: 'flex', alignItems: 'center', gap: '
8px' }}>
                  <button onClick={() =>
                    dispatch(updateQuantity({ id: item._id, quantity: item.
quantity - 1 }))
                  } style={{ background: '#1C2128', color: 'white',
                    border: '1px solid #30363D', borderRadius: '4px',
                    padding: '4px 10px', cursor: 'pointer' }}>
                    -
                  </button>

                  <span style={{ color: 'white', minWidth: '24px',
                    textAlign: 'center' }}>{item.quantity}</span>

                  <button onClick={() =>
                    dispatch(updateQuantity({ id: item._id, quantity: item.
quantity + 1 }))
                  } style={{ background: '#1C2128', color: 'white',
                    border: '1px solid #30363D', borderRadius: '4px',
                    padding: '4px 10px', cursor: 'pointer' }}>
                    +
                  </button>
                </div>
```

```
              {/* Item Total */}
              <p style={{ color: '#56CF8A', fontWeight: 'bold',
                minWidth: '80px', textAlign: 'right' }}>
                Rs. {(item.price * item.quantity).toLocaleString()}
              </p>

              {/* Remove Button */}
              <button
                onClick={() => dispatch(removeItem(item._id))}
                style={{ background: 'none', border: 'none',
                  color: '#FF6B6B', cursor: 'pointer',
                  fontSize: '20px', padding: '4px' }}
              >
                X
              </button>
            </div>
          ))}
        </div>

        {/* ■■ RIGHT: Order Summary ■■■■■■■■■■■■■■■■■■■■ */}
        <div style={{ background: '#161B22', border: '1px solid #30363D',
          borderRadius: '10px', padding: '20px', height: 'fit-content' }}
>
          <h3 style={{ color: '#E6EDF3', marginBottom: '16px' }}>
            Order Summary
          </h3>

          {/* Price rows */}
          {[
            ['Items Total', totalPrice],
            ['Tax (5%)',    taxPrice],
            ['Shipping',   shippingPrice],
          ].map(([label, amount]) => (
            <div key={label} style={{ display: 'flex',
              justifyContent: 'space-between', marginBottom: '10px' }}>
              <span style={{ color: '#8B949E' }}>{label}</span>
```

```
              <span style={{ color: '#E6EDF3' }}>
                Rs. {amount?.toLocaleString(undefined, { minimumFractionD
igits: 0 })}
              </span>
            </div>
          ))}

          {/* Divider */}
          <hr style={{ borderColor: '#30363D', margin: '12px 0' }} />

          {/* Grand Total */}
          <div style={{ display: 'flex', justifyContent: 'space-between',
            marginBottom: '20px' }}>
            <span style={{ color: 'white', fontWeight: 'bold', fontSize:
'16px' }}>
              Grand Total
            </span>
            <span style={{ color: '#56CF8A', fontWeight: 'bold', fontSize
: '18px' }}>
              Rs. {grandTotal?.toLocaleString(undefined, { minimumFractio
nDigits: 0 })}
            </span>
          </div>

          {/* Free shipping notice */}
          {shippingPrice === 0 && (
            <p style={{ color: '#56CF8A', fontSize: '12px',
              marginBottom: '12px', textAlign: 'center' }}>
              ■ Free Shipping!
            </p>
          )}
          {shippingPrice > 0 && (
            <p style={{ color: '#8B949E', fontSize: '12px',
              marginBottom: '12px', textAlign: 'center' }}>
              Rs. 2000+ order pe free shipping milegi
            </p>
          )}
```

```
            {/* Checkout Button */}
            <button onClick={checkoutHandler} style={{
              width:         '100%',
              padding:       '14px',
              background:    '#38BDF8',
              color:         '#0D1117',
              border:        'none',
              borderRadius:  '8px',
              fontSize:      '16px',
              fontWeight:    'bold',
              cursor:        'pointer'
            }}>
              {isLoggedIn ? 'Checkout Karo' : 'Login Karo Checkout Ke Liye'
}
            </button>
          </div>
        </div>
      </div>
    );
}

export default CartScreen;
```

■ TIP

cartSlice mein updateQuantity ka reducer check karo — agar quantity 1 se kam ho toh Math.max(1, quantity) se roka gaya hai. Toh - button press karne pe 0 ya negative quantity nahi aayegi. Remove karna ho toh ✕ button use karo!

SECTION 3 — Rating Component

3.0 — Star Rating Ka Concept

Rating component ek chhota sa reusable piece hai jo stars dikhata hai. Yeh teen props leta hai: **value** (rating number), **text** (optional review count), aur **color** (star color). Yeh ProductCard aur ProductScreen dono mein use hoga:

■ src/components/Rating.jsx

```
// Rating Component
// Props: value (number 0-5), text (optional string), color (optional)
function Rating({ value = 0, text = '', color = '#FFD93D' }) {
  // Helper — ek star ka icon decide karo
  // value = 4.3 hoga toh:
  // star 1 = full (4.3 >= 1)
  // star 2 = full (4.3 >= 2)
  // star 3 = full (4.3 >= 3)
  // star 4 = full (4.3 >= 4)
  // star 5 = half (4.3 >= 4.5? No. 4.3 >= 4? Yes)
  const getStar = (starNumber) => {
    if (value >= starNumber)       return '★';  // Full star
    if (value >= starNumber - 0.5) return '½';  // Half star
    return '■';                                  // Empty star
  };
  return (
    <div style={{ display: 'flex', alignItems: 'center', gap: '4px' }}>
      {/* 5 stars render karo */}
      {[1, 2, 3, 4, 5].map(star => (
        <span
          key={star}
          style={{ color: color, fontSize: '16px' }}
        >
```

```
            {getStar(star)}
          </span>
        ))}
        {/* Optional text — jaise '(128 reviews)' */}
        {text && (
          <span style={{ color: '#8B949E', fontSize: '13px', marginLeft: '6
px' }}>
            {text}
          </span>
        )}
      </div>
    );
}

export default Rating;

// Use karna:
// <Rating value={4.5} text='(128 reviews)' />
// <Rating value={3}   color='#FF6B6B' />
```

SECTION 4 — SearchBox Component

4.0 — Search Kaise Kaam Karega

User search box mein kuch likhe aur Enter dabaye — URL change hoga: **/?keyword=shoes**. HomeScreen URL se keyword padh ke Redux thunk ko bhejega. Server pe products filter honge aur wapas aayenge. Is tarah page refresh pe bhi search result rehta hai!

src/components/SearchBox.jsx

```
import { useState }                    from 'react';
import { useNavigate, useSearchParams } from 'react-router-dom';

function SearchBox() {
  const navigate  = useNavigate();
  const [searchParams] = useSearchParams();

  // URL mein pehle se keyword hai? Toh woh dikhao
  const [keyword, setKeyword] = useState(
    searchParams.get('keyword') || ''
  );

  const submitHandler = (e) => {
    e.preventDefault();  // Page reload rokna

    if (keyword.trim()) {
      // URL mein keyword add karo — HomeScreen padh lega
      navigate(`/?keyword=${keyword.trim()}&page=1`);
    } else {
      // Khaali search — sab products
      navigate('/');
    }
  };

  return (
    <form
      onSubmit={submitHandler}
      style={{ display: 'flex', gap: '8px' }}
```

```
      >
        <input
          type='text'
          value={keyword}
          onChange={e => setKeyword(e.target.value)}
          placeholder='Products dhundho...'
          style={{
            padding:      '8px 14px',
            background:   '#1C2128',
            border:       '1px solid #30363D',
            borderRadius: '6px',
            color:        'white',
            width:        '220px',
            outline:      'none',
          }}
        />
        <button type='submit' style={{
          background:   '#38BDF8',
          color:        '#0D1117',
          border:       'none',
          borderRadius: '6px',
          padding:      '8px 16px',
          fontWeight:   'bold',
          cursor:       'pointer',
        }}>
          ■
        </button>
      </form>
    );
  }

  export default SearchBox;
```

■ TIP

SearchBox ko Navbar mein add karo taake har page pe search available ho. Navbar.jsx mein SearchBox import karo aur logo ke baad render karo.

4.1 — Navbar Mein SearchBox Add Karna

Navbar.jsx ko update karo — SearchBox import karke logo aur links ke beech mein add karo:

■ src/components/Navbar.jsx [SearchBox add karo]

```
// Navbar.jsx mein SearchBox import karo
import SearchBox from './SearchBox';   // Yeh line add karo

// return mein logo aur nav links ke beech:
<Link to='/'>■ ShopEase</Link>

<SearchBox />   {/* Yeh add karo */}

<div>  {/* Nav links */}
  ...
</div>
```

SECTION 5 — HomeScreen — Complete

5.0 — HomeScreen Ka Plan

HomeScreen ab search, category filter, aur pagination ke saath aayega. URL se keyword aur page number padh ke Redux thunk call hoga:

■ src/screens/HomeScreen.jsx [Complete version]

```
import { useEffect }                    from 'react';
import { useDispatch, useSelector }      from 'react-redux';
import { useSearchParams }               from 'react-router-dom';

import { fetchProducts, selectProducts,
         selectProductsLoading,
         selectProductsError,
         selectTotalPages }              from '../store/productSlice';

import ProductCard  from '../components/ProductCard';
import Paginate     from '../components/Paginate';
import Loader       from '../components/Loader';
import Message      from '../components/Message';

// Categories list
const CATEGORIES = ['All','Footwear','Clothing','Electronics','Books','Bags','Other'];

function HomeScreen() {
  const dispatch = useDispatch();

  // URL se keyword, category aur page lo
  const [searchParams, setSearchParams] = useSearchParams();
  const keyword  = searchParams.get('keyword')  || '';
  const category = searchParams.get('category') || '';
  const page     = Number(searchParams.get('page')) || 1;

  // Redux store se state
  const products   = useSelector(selectProducts);
  const loading    = useSelector(selectProductsLoading);
```

```
  const error      = useSelector(selectProductsError);
  const totalPages = useSelector(selectTotalPages);

  // Jab bhi keyword, category ya page badle — products fetch karo
  useEffect(() => {
    dispatch(fetchProducts({ keyword, page, category }));
  }, [dispatch, keyword, page, category]);

  // Category filter click handler
  const categoryHandler = (cat) => {
    const params = {};
    if (cat !== 'All') params.category = cat;
    if (keyword)       params.keyword  = keyword;
    params.page = 1;  // Category badli toh page 1 se shuru
    setSearchParams(params);
  };

  return (
    <div style={{ maxWidth: '1200px', margin: '0 auto', padding: '24px' }
}>

      {/* ■■ Category Filter ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */}
      <div style={{ display: 'flex', gap: '8px',
        flexWrap: 'wrap', marginBottom: '24px' }}>
        {CATEGORIES.map(cat => (
          <button
            key={cat}
            onClick={() => categoryHandler(cat)}
            style={{
              padding:      '6px 16px',
              borderRadius: '20px',
              border:       '1px solid #30363D',
              cursor:       'pointer',
              fontWeight:   category === cat || (cat==='All' && !category
) ? 'bold':'normal',
              background:   category === cat || (cat==='All' && !category
)
```

```
                ? '#38BDF8' : '#1C2128',
              color:       category === cat || (cat==='All' && !category
)
                ? '#0D1117' : '#8B949E',
            }}
          >
            {cat}
          </button>
        ))}
      </div>

      {/* ■■ Search Result Heading ■■■■■■■■■■■■■■■■■■■■ */}
      {keyword && (
        <h2 style={{ color: '#E6EDF3', marginBottom: '16px' }}>
          '{keyword}' ke results
        </h2>
      )}

      {/* ■■ Loading / Error / Products ■■■■■■■■■■■■■■■■ */}
      {loading ? (
        <Loader />
      ) : error ? (
        <Message type='error'>{error}</Message>
      ) : products.length === 0 ? (
        <Message type='info'>Koi product nahi mila!</Message>
      ) : (
        <>
          {/* Products Grid */}
          <div style={{
            display:             'grid',
            gridTemplateColumns: 'repeat(auto-fill, minmax(230px, 1fr))',
            gap:                 '20px',
            marginBottom:        '32px',
          }}>
            {products.map(product => (
```

```
              <ProductCard key={product._id} product={product} />
            ))}
          </div>

          {/* Pagination */}
          {totalPages > 1 && (
            <Paginate
              page={page}
              totalPages={totalPages}
              keyword={keyword}
              category={category}
            />
          )}
        </>
      )}
    </div>
  );
}

export default HomeScreen;
```

SECTION 6 — Loader, Message aur ProductCard

6.0 — Loader Component

■ src/components/Loader.jsx

```
// Loader — API call ke dauran dikhao
function Loader() {
  return (
    <div style={{ textAlign: 'center', padding: '40px' }}>
      {/* CSS spinner — keyframes CSS file mein add karo */}
      <div style={{
        width:        '48px',
        height:       '48px',
        border:       '4px solid #30363D',
        borderTop:    '4px solid #38BDF8',
        borderRadius: '50%',
        animation:    'spin 0.8s linear infinite',
        margin:       '0 auto',
      }} />
      <p style={{ color: '#8B949E', marginTop: '12px' }}>Loading...</p>
    </div>
  );
}
export default Loader;

// index.css ya App.css mein yeh add karo:
// @keyframes spin {
//   from { transform: rotate(0deg); }
//   to   { transform: rotate(360deg); }
// }
```

6.1 — Message Component

■ src/components/Message.jsx

```
// Message — success, error, info dikhao
function Message({ type = 'info', children }) {
  const styles = {
    error:   { background: '#2D1515', color: '#FF6B6B', border: '1px soli
d #FF6B6B' },
    success: { background: '#0A2818', color: '#56CF8A', border: '1px soli
d #56CF8A' },
    info:    { background: '#0D2137', color: '#79C0FF', border: '1px soli
d #388BFD' },
  };
  return (
    <div style={{
      ...styles[type],
      padding:      '12px 16px',
      borderRadius: '8px',
      margin:       '12px 0',
    }}>
      {children}
    </div>
  );
}
export default Message;

// Use karna:
// <Message type='error'>Kuch galat ho gaya!</Message>
// <Message type='success'>Order place ho gaya!</Message>
// <Message type='info'>Koi product nahi mila</Message>
```

6.2 — ProductCard — Refined Version

■ src/components/ProductCard.jsx [Rating + Quick Add]

```
import { Link }   from 'react-router-dom';
import { useDispatch } from 'react-redux';
import Rating     from './Rating';
```

```
import { addItem } from '../store/cartSlice';

function ProductCard({ product }) {
  const dispatch = useDispatch();
  const { _id, name, images, price, rating, numReviews, stock } = product
;

  const quickAddHandler = (e) => {
    e.preventDefault();  // Link ke andar hai — navigate mat karo
    dispatch(addItem({
      _id, name, price, stock,
      image:    images?.[0] || '/images/default.jpg',
      quantity: 1,
    }));
  };

  return (
    <Link to={`/product/${_id}`} style={{ textDecoration: 'none' }}>
      <div style={{
        background:   '#161B22',
        border:       '1px solid #30363D',
        borderRadius: '12px',
        overflow:     'hidden',
        transition:   'transform 0.2s, border-color 0.2s',
      }}
        onMouseEnter={e => {
          e.currentTarget.style.transform    = 'translateY(-4px)';
          e.currentTarget.style.borderColor  = '#38BDF8';
        }}
        onMouseLeave={e => {
          e.currentTarget.style.transform    = 'translateY(0)';
          e.currentTarget.style.borderColor  = '#30363D';
        }}
      >
        {/* Image */}
        <img
```

```
            src={images?.[0] || '/images/default.jpg'}
            alt={name}
            style={{ width: '100%', height: '200px', objectFit: 'cover' }}
          />

          <div style={{ padding: '16px' }}>
            {/* Naam */}
            <h3 style={{ color: '#E6EDF3', fontSize: '14px',
              marginBottom: '8px', lineHeight: '1.4' }}>
              {name}
            </h3>

            {/* Rating */}
            <div style={{ marginBottom: '10px' }}>
              <Rating
                value={rating}
                text={`(${numReviews})`}
              />
            </div>

            {/* Price aur Stock */}
            <div style={{ display: 'flex',
              justifyContent: 'space-between', alignItems: 'center' }}>
              <p style={{ color: '#56CF8A', fontSize: '18px', fontWeight: '
bold' }}>
                Rs. {price?.toLocaleString()}
              </p>
              <span style={{
                fontSize:     '11px',
                padding:      '2px 8px',
                borderRadius: '12px',
                background:   stock > 0 ? '#0A2818' : '#2D1515',
                color:        stock > 0 ? '#56CF8A' : '#FF6B6B',
              }}>
                {stock > 0 ? 'In Stock' : 'Out of Stock'}
              </span>
```

```
            </div>

            {/* Quick Add Button */}
            {stock > 0 && (
              <button
                onClick={quickAddHandler}
                style={{
                  width:        '100%',
                  marginTop:    '12px',
                  padding:      '8px',
                  background:   '#1C2128',
                  color:        '#38BDF8',
                  border:       '1px solid #38BDF8',
                  borderRadius: '6px',
                  cursor:       'pointer',
                  fontSize:     '13px',
                }}
              >
                + Quick Add
              </button>
            )}
          </div>
        </div>
      </Link>
    );
  }

  export default ProductCard;
```

SECTION 7 — Paginate Component

7.0 — Pagination Ka Concept

Agar 100 products hain toh ek baar mein sab load karna slow hai. Pagination se 12-12 products pages mein divide hote hain. URL mein page number hota hai — **/?page=2** — taake browser back button se pichli page pe wapas jaa sake:

src/components/Paginate.jsx

```
import { useNavigate } from 'react-router-dom';

function Paginate({ page, totalPages, keyword = '', category = '' }) {
  const navigate = useNavigate();

  // Page change karna
  const goToPage = (pageNum) => {
    const params = new URLSearchParams();
    params.set('page', pageNum);
    if (keyword)  params.set('keyword',  keyword);
    if (category) params.set('category', category);
    navigate(`/?${params.toString()}`);
  };

  // Sirf 1 page hai toh mat dikhao
  if (totalPages <= 1) return null;

  // Page numbers array banao — [1, 2, 3, ... totalPages]
  const pages = Array.from({ length: totalPages }, (_, i) => i + 1);

  return (
    <div style={{
      display:        'flex',
      justifyContent: 'center',
      gap:            '8px',
      margin:         '24px 0'
    }}>

      {/* Previous button */}
```

```
<button
  onClick={() => goToPage(page - 1)}
  disabled={page === 1}
  style={{
    padding:      '8px 14px',
    borderRadius: '6px',
    border:       '1px solid #30363D',
    background:   page === 1 ? '#0D1117' : '#1C2128',
    color:        page === 1 ? '#30363D' : '#8B949E',
    cursor:       page === 1 ? 'not-allowed' : 'pointer',
  }}
>
  &larr;
</button>

{/* Page numbers */}
{pages.map(p => (
  <button
    key={p}
    onClick={() => goToPage(p)}
    style={{
      padding:      '8px 14px',
      borderRadius: '6px',
      border:       '1px solid #30363D',
      background:   p === page ? '#38BDF8' : '#1C2128',
      color:        p === page ? '#0D1117'  : '#8B949E',
      fontWeight:   p === page ? 'bold'     : 'normal',
      cursor:       'pointer',
    }}
  >
    {p}
  </button>
))}

{/* Next button */}
```

```
        <button
          onClick={() => goToPage(page + 1)}
          disabled={page === totalPages}
          style={{
            padding:      '8px 14px',
            borderRadius: '6px',
            border:       '1px solid #30363D',
            background:   page === totalPages ? '#0D1117' : '#1C2128',
            color:        page === totalPages ? '#30363D' : '#8B949E',
            cursor:       page === totalPages ? 'not-allowed' : 'pointer',
          }}
        >
          &rarr;
        </button>
      </div>
    );
  }

  export default Paginate;
```

SECTION 8 — Final File Structure

8.0 — Chapter 1 Se 9 Tak Ki Saari Files

Yeh complete reference hai — kaunsi file kahan hogi aur usne kab bani:

Backend Files (Chapters 1-5)

File Path	Chapter	Kya Karta Hai?
`server.js`	2,3,4	Entry point — middleware, routes, server start
`.env`	3	Secret keys — MONGO_URI, JWT_SECRET, PORT
`.gitignore`	1	node_modules, .env ignore karna
`package.json`	1	Project config — scripts, dependencies
`config/db.js`	3	MongoDB Atlas connection
`config/config.js`	4	Centralized env variables
`models/Product.js`	3	Product schema — naam, price, images, reviews
`models/User.js`	3,5	User schema — bcrypt hash, comparePassword, generateToken
`models/Order.js`	3	Order schema — items, payment, delivery
`controllers/productController.js`	4	getProducts, getProductById, create, update, delete, review
`controllers/userController.js`	4,5	register, login, getProfile, updateProfile
`controllers/orderController.js`	4	createOrder, getMyOrders, getOrderById, updateStatus
`middleware/asyncHandler.js`	4	try-catch wrapper — DRY
`middleware/errorMiddleware.js`	4	Global error handler, 404, AppError class
`middleware/authMiddleware.js`	5	protect, admin middlewares
`routes/productRoutes.js`	4,5	Product endpoints — auth applied
`routes/userRoutes.js`	4,5	User endpoints — protect applied
`routes/orderRoutes.js`	4,5	Order endpoints — protect + admin

Frontend Files (Chapters 6-9)

File Path	Chapter	Kya Karta Hai?
`frontend/src/index.js`	6	Entry point — Provider, BrowserRouter wrap

frontend/src/App.js	7	All routes define — ProtectedRoute apply
frontend/src/index.css	6	Global CSS — @keyframes spin
frontend/src/store/store.js	8	Redux store — saare reducers combine
frontend/src/store/cartSlice.js	8	Cart — addItem, removeItem, updateQty, clear
frontend/src/store/authSlice.js	8	Auth — setCredentials, logout
frontend/src/store/productSlice.js	8	Products — fetchProducts, fetchById thunks
frontend/src/services/api.js	6	Axios instance — interceptor, API functions
frontend/src/components/Navbar.jsx	7,9	Navigation — SearchBox, cart badge, auth
frontend/src/components/SearchBox.jsx	9	Search input — URL keyword update
frontend/src/components/ProductCard.jsx	6,9	Product card — Rating, Quick Add
frontend/src/components/Rating.jsx	9	Star rating display
frontend/src/components/Loader.jsx	9	Spinning loader
frontend/src/components/Message.jsx	9	Error/success/info message
frontend/src/components/Paginate.jsx	9	Page navigation buttons
frontend/src/components/ProtectedRoute.jsx	7	Login check — redirect to /login
frontend/src/screens/HomeScreen.jsx	6,9	Products grid — search, filter, pagination
frontend/src/screens/ProductScreen.jsx	7,9	Product detail — quantity, add to cart FIXED
frontend/src/screens/CartScreen.jsx	9	Cart — items, totals, checkout NEW
frontend/src/screens/LoginScreen.jsx	8	Login form — dispatch setCredentials
frontend/src/screens/RegisterScreen.jsx	7	Register form
frontend/src/screens/ProfileScreen.jsx	7	User profile — update naam, password

SECTION 9 — Summary, Practice & Next Steps

9.0 — Chapter 9 Ka Recap

■	**Gap Fix — ProductScreen**	addItem dispatch karna — Redux cartSlice se connected
■	**Gap Fix — CartScreen**	Poori file likhi — items, qty update, remove, total
■	**Rating Component**	Star display — value, text, color props
■	**SearchBox**	URL mein keyword — navigate se search
■	**HomeScreen**	Search + category filter + pagination — complete
■	**ProductCard**	Rating + hover + Quick Add button
■	**Loader**	Spinning CSS animation — API ke dauran
■	**Message**	type prop — error/success/info colors
■	**Paginate**	Page buttons — prev/next, active highlight
■	**File Reference**	Backend 18 files + Frontend 22 files — sab listed

9.1 — Practice Exercises

■ Easy Level

1. ProductScreen open karo — Add to Cart click karo — Navbar mein badge check karo
2. CartScreen open karo — item remove karo — total update check karo
3. Rating component import karo — 3.5 value pass karo
4. SearchBox mein 'shoes' likho — Enter dabao — URL check karo

■ Medium Level

1. HomeScreen mein Category filter test karo — 'Footwear' click karo
2. 10+ products banao backend mein — Paginate component test karo
3. Message component — error, success, info teeno test karo
4. CartScreen mein shipping free hone ka test karo (2000+ order)

■ Challenge Level

1. ProductCard ka hover effect aur Quick Add button test karo
2. Search + Category dono ek saath use karo — URL dekho
3. Empty cart message test karo — Shopping Karo link check karo
4. Logout karo — Cart mein jao — Checkout button 'Login Karo' dikhata hai?

9.2 — Chapter 10 Ka Preview

Checkout, Orders & Payment Integration

- ■ CheckoutScreen — shipping address form
- ■ PaymentScreen — payment method select karna
- ■ OrderSummaryScreen — final review before placing
- ■ Order place karna — backend API call
- ■ OrdersScreen — apne saare orders dekhna
- ■ Admin — OrdersScreen — saare orders manage karna
- ■ Razorpay/Stripe payment gateway integration
- ■ Order status update — Processing to Delivered

Shukar Hai — Ab App Chal Raha Hai! ■

Chapter 8 ke gaps fix ho gaye — ProductScreen aur CartScreen dono working hain. HomeScreen mein search, filter, pagination sab aa gaya. Aur ab tumhare paas 40 files ka complete reference bhi hai. **Chapter 10 mein checkout aur payment add karenge — phir sirf deployment bacha hoga!**